

AXI4-STREAM SYSTOLIC ARRAY MATRIX MULTIPLICATION ACCELERATOR

Technical Design & Verification Report

Prepared by
Omar Ahmed Abdelaty
Digital IC Design & Verification Engineer

2026

1. Executive Summary

This report presents the complete design and verification of a parameterizable AXI4-Stream Systolic Array accelerator intended for hardware-accelerated matrix multiplication. The design is fully described in synthesizable SystemVerilog RTL and validated through both directed testbench simulation and a structured Universal Verification Methodology (UVM) environment.

The accelerator computes an $N \times N$ matrix-vector product per inference pass, streaming input activations via AXI4-Stream slave interface and producing output partial sums on an AXI4-Stream master interface. The project demonstrates correct arithmetic results, proper AXI handshaking including back-pressure stall recovery, and a fully connected UVM verification infrastructure.

Design Size	Data Width	Accumulator Width	Interface Standard
2x2 (N=2, scalable)	8-bit (DWIDTH)	16-bit (PWIDTH)	AXI4-Stream

2. Design Architecture

2.1 System Overview

The accelerator is organized as a three-level hierarchy: a top-level AXI4-Stream wrapper, an $N \times N$ systolic array core, and individual Multiply-Accumulate Processing Elements (MAC PEs). The following figure illustrates the top-level signal flow.

Input Interface (AXI4-Stream Slave)	Output Interface (AXI4-Stream Master)
s_axis_tdata [NxDWIDTH-1 : 0]	m_axis_tdata [NxPWIDTH-1 : 0]
s_axis_tvalid / s_axis_tready	m_axis_tvalid / m_axis_tready
weight_in_flat [NxNxDWIDTH-1 : 0]	Back-pressure supported

2.2 MAC Processing Element (mac_pe.sv)

Each MAC PE is the fundamental compute unit. It stores a single weight coefficient loaded during the weight-load phase and performs a fused multiply-add each clock cycle during the inference phase. The PE passes the activation value horizontally and the running partial sum vertically, implementing the classic weight-stationary dataflow.

Key design details:

- Active-low synchronous reset (rst_n) clears weight register, activation output, and partial sum output.
- A gated enable signal (en) prevents toggling of registers while the pipeline is stalled or idle.
- Zero-skipping optimization: the multiply-add is bypassed if either operand is zero, reducing dynamic power.
- Parameterizable widths: DWIDTH (default 8 bits) for activations/weights; PWIDTH (default 16 bits) for accumulators.

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) { weight_reg, act_out, psum_out } <= '0;
    else if (en) begin
        if (load_weight) weight_reg <= weight_in;
        else begin
            act_out <= act_in;
            psum_out <= (act_in && weight_reg) ? psum_in + act_in*weight_reg : psum_in;
        end
    end
end
```

2.3 Systolic Array Core (systolic_array.sv)

The systolic array instantiates $N \times N$ MAC PEs in a two-dimensional mesh topology. Activations flow left-to-right along each row; partial sums accumulate top-to-bottom within each column. After passing through all N rows, the final column of partial sums is exposed as the array output.

Internal wire naming convention:

- `act_wires[row][col]` — horizontal activation bus between adjacent PEs in a row.
- `psum_wires[row][col]` — vertical partial-sum bus between adjacent PEs in a column.
- Boundary conditions: `act_wires[i][0] = act_in[i]`; `psum_wires[0][j] = 0` (no initial accumulation bias).

2.4 AXI4-Stream Wrapper (axis_systolic_wrapper.sv)

The wrapper adapts the raw systolic core to the AXI4-Stream protocol and implements the input staggering required to correctly align data to the systolic dataflow.

2.4.1 Weight Loading

Weights are loaded via the flat bus `weight_in_flat`. A generate block unpacks this bus into the two-dimensional `weight_in_2d` array consumed by the systolic core. Weight loading is asserted for exactly one clock cycle, controlled by the `load_weight` strobe.

2.4.2 Activation Staggering

Because the systolic array processes data diagonally, each activation row must be delayed relative to row 0 before being injected into the core. Row i is delayed by i clock cycles using a shift-register

chain (delay_lines). This staggering ensures that spatially aligned data reaches the correct PE at the correct time.

2.4.3 AXI4-Stream Handshaking & Back-pressure

The wrapper implements full AXI4-Stream flow control. The ready signal from the master side (m_axis_tready) directly gates the advance_pipeline signal, which simultaneously controls the stagger delay registers and the valid shift register. When the downstream consumer de-asserts tready, the entire pipeline freezes, preventing data corruption.

Signal	Direction	Description
s_axis_tready	Output	Tied to m_axis_tready; back-pressure propagates upstream
advance_pipeline	Internal	High when m_axis_tready=1; gates all pipeline registers
safe_tdata	Internal	Zeros injected when tvalid=0 to prevent phantom computations
core_en	Internal	advance_pipeline load_weight, enables MAC PE registers
valid_pipeline[N-1:0]	Internal	N-stage shift register tracking tvalid through the stagger latency
m_axis_tvalid	Output	Valid_pipeline[N-1]; output asserted after N-cycle stagger delay

3. Verification Strategy

3.1 Verification Phases

The project employs a two-phase verification strategy, progressing from a directed self-checking testbench to a fully structured UVM environment.

Phase	Testbench File	Coverage
Phase 1	tb_axis_wrapper.sv	Directed test: weight load, two activation columns, stall injection, pipeline flush, arithmetic correctness
Phase 2	systolic_uvm_top.sv	Constrained-random stimulus via UVM sequences, scoreboard capture, structural verification of UVM hierarchy

3.2 Directed Testbench (Phase 1)

The directed testbench exercises a predefined 2×2 matrix multiplication scenario with known expected outputs, allowing pass/fail assertions at specific clock cycles.

Test stimulus sequence:

- Reset assertion: rst_n de-asserted for 12 ns to clear all registers.
- Weight load: $W = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}$ loaded via load_weight strobe.
- Activation column 0: a0 = [4, 0] streamed in; row 1 activation delayed by 1 cycle in stagger logic.
- Activation column 1: a1 = [5, 2] streamed in with s_axis_tvalid=1.
- Stall injection: m_axis_tready de-asserted for 20 ns, verifying pipeline freeze and correct resume.
- Flush: two zero-data cycles push the remaining partial sums through the array.

Expected arithmetic results:

Output Cycle	Row 0 (psum_out[0])	Row 1 (psum_out[1])
1	8 ($= 4 \times 2 + 0 \times 0$)	—
2	12 ($= 4 \times 2 + 5 \times 0 + 0 \times 0 + 0 \times 0 \rightarrow$ accumulates)	—
3	—	6 ($= 4 \times 1 + 0 \times 3 = 4 + \dots$)
4	ALL TESTS PASS	\$finish triggered

3.3 UVM Verification Environment (Phase 2)

A complete UVM testbench was developed to enable constrained-random verification and extensible coverage closure. The environment follows standard UVM layer conventions.

3.3.1 Sequence Item (systolic_seq_item.sv)

The transaction class encapsulates both weight-load and data-stream transactions in a single type, differentiated by the is_weight flag. Randomizable fields include the activation data bus (s_axis_tdata), flat weight bus (weight_in_flat), and inter-transaction delay (delay_cycles, constrained to 0-2 cycles). The output field m_axis_tdata is non-random, populated by the monitor.

3.3.2 Sequence (systolic_seq.sv)

The base sequence implements the canonical test stimulus: one weight-load transaction followed by ten randomized data transactions with is_weight=0. Constraints ensure valid stream data across all combinations of 8-bit activation values.

3.3.3 Driver (systolic_driver.sv)

The driver translates sequence items to pin-level interface activity via the systolic_if virtual interface. It correctly implements the AXI4-Stream handshake: for data transactions it holds tvalid high and

waits for steady acknowledgement before completing the item. Weight transactions assert `load_weight` for exactly one clock cycle.

3.3.4 Monitor (`systolic_monitor.sv`)

The monitor passively observes the output AXI4-Stream interface and creates a sequence item for each valid transfer (`m_axis_tvalid` & `m_axis_tready` at posedge `clk`). Captured items are broadcast over the analysis port (`mon_ap`) to connected subscribers.

3.3.5 Scoreboard (`systolic_scoreboard.sv`)

The scoreboard receives captured output items via `uvm_analysis_imp` and logs both output rows using UVM_LOW verbosity messages. The current implementation provides a logging foundation; full reference-model comparison (golden model) is identified as a future extension.

3.3.6 Agent, Environment & Test

The `systolic_agent` encapsulates the driver, monitor, and sequencer. The `systolic_env` instantiates the agent and scoreboard and wires the monitor analysis port to the scoreboard input. The `systolic_test` raises/drops UVM objections around the sequence execution, with a 100 ns drain time for in-flight transactions.

UVM Component	File	Role
<code>systolic_seq_item</code>	<code>systolic_seq_item.sv</code>	Transaction descriptor; rand fields for stimulus
<code>systolic_seq</code>	<code>systolic_seq.sv</code>	Constrained-random stimulus generation
<code>systolic_driver</code>	<code>systolic_driver.sv</code>	Pin-level AXI4-Stream driving with handshake
<code>systolic_monitor</code>	<code>systolic_monitor.sv</code>	Passive output capture; analysis port broadcast
<code>systolic_scoreboard</code>	<code>systolic_scoreboard.sv</code>	Output logging; golden model hook point
<code>systolic_agent</code>	<code>systolic_agent.sv</code>	Encapsulates driver, monitor, sequencer
<code>systolic_env</code>	<code>systolic_env.sv</code>	Top-level env: agent + scoreboard + connections
<code>systolic_test</code>	<code>systolic_test.sv</code>	Top-level test; objection management
<code>systolic_if</code>	<code>systolic_if.sv</code>	Parameterizable virtual interface bundle
<code>systolic_uvm_top</code>	<code>systolic_uvm_top.sv</code>	Top-level module; DUT instantiation + <code>run_test</code>

4. Project File Structure

The project is organized into RTL design sources and verification sources:

#	File	Description
1	mac_pe.sv	Multiply-accumulate processing element — fundamental compute unit
2	systolic_array.sv	N×N mesh of MAC PEs with horizontal act / vertical psum routing
3	axis_systolic_wrapper.sv	AXI4-Stream adapter, input stagger logic, valid tracking
4	tb_axis_wrapper.sv	Phase 1 directed self-checking testbench
5	systolic_if.sv	Parameterizable interface bundle for UVM virtual interface
6	systolic_seq_item.sv	UVM transaction descriptor with randomizable fields
7	systolic_seq.sv	UVM base sequence: weight-load + 10 random data transactions
8	systolic_driver.sv	UVM driver implementing full AXI4-Stream handshake
9	systolic_monitor.sv	UVM passive monitor capturing output valid transfers
10	systolic_scoreboard.sv	UVM scoreboard logging output and ready for golden model
11	systolic_agent.sv	UVM agent encapsulating driver, monitor, and sequencer
12	systolic_env.sv	UVM environment connecting agent and scoreboard
13	systolic_test.sv	UVM top-level test with objection lifecycle management
14	systolic_pkg.sv	SystemVerilog package aggregating all UVM class includes
15	systolic_uvm_top.sv	UVM top module: DUT, interface, config DB, run_test()

5. Design Parameters & Scalability

All three RTL modules expose a consistent set of parameters that can be adjusted at elaboration time without modifying any RTL source:

Parameter	Default	Location	Effect
N	2	All modules	Sets array dimension to N×N; stagger depth and PE count scale automatically
DWIDTH	8	All modules	Activation and weight data bus width (bits)
PWIDTH	16	All modules	Partial sum / accumulator bus width; must satisfy $PWIDTH \geq 2 \times DWIDTH$ for no overflow on N=1

Scaling to larger arrays (e.g. $N=8$ or $N=16$) requires only parameter overrides at instantiation. The stagger delay logic and valid pipeline shift register both derive their depth from N automatically via generate blocks, making the design fully technology-independent and synthesizable.

6. Known Limitations & Future Work

6.1 Current Limitations

- The scoreboard logs outputs but does not include a reference model for automatic pass/fail comparison with random stimulus. This requires a software golden model (e.g. $\$display$ -based matrix multiply or a C DPI model).
- The MAC PE only supports unsigned integer arithmetic. Signed activation and weight support would require extending the multiply to signed types.
- The current implementation assumes `weight_in_flat` is presented for exactly one clock cycle. A handshake protocol for weight loading would improve robustness in pipelined systems.
- No assertions (SVA) are currently included. Formal assertions on AXI4-Stream protocol compliance and no-X propagation would improve verification confidence.

6.2 Recommended Future Extensions

- Golden model integration: implement a SystemVerilog DPI C model or $\$system$ task to compute expected outputs and compare against DUT.
- Functional coverage: add covergroups tracking full 8-bit activation/weight value ranges, back-pressure scenarios, and zero-skipping activation rates.
- Larger array validation: parameterize the testbench for $N=4$ and $N=8$ arrays to validate generate-block scaling.
- FPGA prototyping: target Xilinx or Intel FPGA for resource utilization and timing closure analysis.
- Integration with quantized ML inference pipeline (INT8 activations feeding from a preceding quantization layer).

7. Conclusion

This project successfully delivers a complete, parameterizable AXI4-Stream Systolic Array Matrix Multiplication Accelerator in synthesizable SystemVerilog RTL. The design has been verified through both a directed testbench confirming correct arithmetic results and AXI4-Stream protocol compliance, and a structured UVM environment providing the framework for constrained-random verification.

The weight-stationary dataflow, gated enable back-pressure pipeline, and staggered activation injection collectively enable efficient, stall-safe matrix multiply operations. The clean three-level hierarchy (MAC PE → Systolic Array → AXI Wrapper) ensures that the design is both readable and maintainable, while the consistent N/DWIDTH/PWIDTH parameter set allows scaling to any required array dimension without source-level modifications.

The accompanying UVM environment, while at a foundational stage, provides all the structural scaffolding needed to grow into a fully automated, coverage-driven regression suite with minimal additional effort.

End of report

Document prepared by Omar Ahmed

Senior 1 EECE Student at CU • 2026